Selected R Analysis Implementation

Supplemental code appendix | GSE297945 and LMEX0000004400

Scope and reproducibility. Curated, non-standalone excerpts from the original analysis script. Code is preserved as supplied, with publication graphics, some intermediate analyses, and setup dependencies omitted. For exact reproduction, use the complete source script and required inputs which will be available upon manuscript acceptance.

0A. PACKAGES

```
suppressPackageStartupMessages({
  library(Seurat);  library(Matrix)
  library(dplyr);   library(tidyr);  library(purrr);  library(tibble)
  library(ggplot2); library(patchwork); library(ggrepel); library(ggtext)
})

for (pkg in c("RANN", "vegan", "DESeq2", "ggrastr", "MASS", "ggtext")) {
  if (!requireNamespace(pkg, quietly = TRUE)) stop("Please install the ", pkg, " package")
}

set.seed(42)
```

0C. COHORT (single source of truth)

```
REF_DONORS     <- c("PDL001", "PDL002", "PDL003", "PDL005", "PDL006",
                    "PDL010", "PDL011", "PDL012", "PDL013")
INJURED_DONORS <- c("PDL004", "PDL007", "PDL008", "PDL009")   # source "Preterm - injured"
RARE_DONORS    <- c("PDL014", "PDL015", "PDL016")             # CHAOS; pulmonary hypoplasia; ARB-fetopathy BPD
SENS_INJURED_DONORS <- c(INJURED_DONORS, "PDL016")
COHORT_DONORS  <- c(REF_DONORS, INJURED_DONORS, RARE_DONORS)

stopifnot(length(REF_DONORS) == 9, length(INJURED_DONORS) == 4,
          length(RARE_DONORS) == 3, !anyDuplicated(COHORT_DONORS))

ref_role  <- "Developmental reference"
inj_role  <- "Preterm injured"
rare_role <- "Rare disease"
role_levels <- c(ref_role, inj_role, rare_role)

role_of <- function(d) factor(dplyr::case_when(
  d %in% REF_DONORS     ~ ref_role,
  d %in% INJURED_DONORS ~ inj_role,
  d %in% RARE_DONORS    ~ rare_role), levels = role_levels)
```

0D. POPULATIONS

```
pop_levels  <- c("Macrophage", "Monocyte", "Dendritic cell", "T cell", "NK", "B cell")
comp_levels <- c(pop_levels, "Other")
mac_levels  <- c("Alveolar macrophage", "SPP1+ macrophage")
axis_levels <- c("Global", pop_levels, mac_levels)

broad_population <- function(ct) dplyr::case_when(
  ct %in% c("Alveolar macrophages", "SPP1+ macrophages") ~ "Macrophage",
  ct %in% c("Monocytes", "Proliferating monocytes")      ~ "Monocyte",
  ct %in% c("cDC", "pDC")                                ~ "Dendritic cell",
  ct %in% c("T", "Treg")                                 ~ "T cell",
  ct == "NK & NKT"                                       ~ "NK",
  ct == "B"                                              ~ "B cell")
```

```
mac_subtype <- function(ct) dplyr::case_when(
  ct == "Alveolar macrophages" ~ "Alveolar macrophage",
  ct == "SPP1+ macrophages"    ~ "SPP1+ macrophage")
```

0E. ANALYSIS PARAMETERS

```
MIN_CELLS       <- 50      # donor x population cells needed to report a score
NONIMMUNE_RATIO <- 0.2     # immune CPM / max non-immune CPM below this = non-immune gene
MIN_LOGCPM      <- 5       # off-path: mean reference log2 CPM for a gene to be scored
N_PERM_PC       <- 20000   # selection-aware PC permutation
N_PERM_ADON     <- 9999    # PERMANOVA / dispersion
CYCLE_GENES     <- c("MKI67", "TOP2A", "CENPF", "CDK1", "CCNB2")
N_GENES_2B      <- 12      # genes per direction in Figure 2B

# Niche-class rules applied to each source CNiche's lineage composition (in order)
EPI_MIN    <- 0.50
IMMUNE_MIN <- 0.15
ENDO_MIN   <- 0.40
niche_class_levels <- c("Epithelial", "Immune-enriched", "Vascular", "Mesenchymal")
lin_levels <- c("Epithelial", "Mesenchymal", "Endothelial", "Immune")

SCALE_BAR_UM <- 1000       # scale bars on tissue maps; centroids assumed in microns (Xenium default)

pred_limit <- stats::qt(0.975, df = length(REF_DONORS) - 2)   # 95% prediction limit, +/-2.36
```

0H. SHARED STATISTICAL HELPERS

```
pseudobulk <- function(counts, groups) {                  # genes x cells -> genes x groups
  g <- factor(groups)
  X <- counts %*% Matrix::sparse.model.matrix(~ 0 + g)
  colnames(X) <- levels(g)
  as.matrix(X)
}
logcpm <- function(x) log2(t(t(x) / colSums(x)) * 1e6 + 1)

counts_for <- function(pb, label) {                       # donor columns for one population
  cols <- colnames(pb)[endsWith(colnames(pb), paste0("___", label))]
  x <- pb[, cols, drop = FALSE]; colnames(x) <- sub("___.*", "", cols); x
}

# Reference model for any donor-level score (column Developmental_PC).
# Deviation = (observed - expected) / prediction SE (residual scatter plus
# uncertainty of the fitted line). Reference donors are scored leave-one-out.
# `ref` names the reference role (Xenium reference or LungMAP control).
fit_reference  <- function(ref_df) stats::lm(Developmental_PC ~ PMA_weeks, data = ref_df)
deviation_score <- function(model, df) {
  p <- stats::predict(model, newdata = df, se.fit = TRUE)
  (df$Developmental_PC - p$fit) / sqrt(p$se.fit^2 + p$residual.scale^2)
}
predict_band <- function(model, x) {
  p <- stats::predict(model, newdata = data.frame(PMA_weeks = x),
                      interval = "prediction", level = 0.95)
  tibble(PMA_weeks = x, fit = p[, "fit"], lwr = p[, "lwr"], upr = p[, "upr"])
}
score_with_loo <- function(df, ref = ref_role) {
  is_ref <- df$role == ref
  m <- fit_reference(df[is_ref, ])
  df$expected  <- stats::predict(m, newdata = df)
  df$dev_score <- deviation_score(m, df)
  for (i in which(is_ref)) {
    df$dev_score[i] <- deviation_score(fit_reference(df[is_ref & seq_len(nrow(df)) != i, ]), df[i, ])
  }
```

```
  rng <- range(df$PMA_weeks[is_ref])
  df$extrapolated <- !is_ref & (df$PMA_weeks < rng[1] | df$PMA_weeks > rng[2])
  n_ref <- sum(is_ref)                                    # 95% limits: comparators n-2 df, LOO reference n-3 df
  df$limit   <- ifelse(is_ref, stats::qt(0.975, n_ref - 3), stats::qt(0.975, n_ref - 2))
  df$exceeds <- abs(df$dev_score) > df$limit
  df
}
add_point_style <- function(df) {                        # within range filled; extrapolated hollow
  df %>% mutate(role_hex = unname(role_colors[as.character(role)]),
                pt_fill  = if_else(extrapolated, "white", role_hex),
                pt_col   = if_else(extrapolated, role_hex, "white"))
}


# SECTION 1 - DATA----
```

1A. CELL METADATA AND DONOR TABLE

```
df <- GSE297945_final_complete_cniches_tniches_allsamples_2025_05_26
stopifnot(inherits(df, "Seurat"))

meta <- df[[]] %>%
  as_tibble(rownames = "cell") %>%
  transmute(cell,
            donor   = as.character(Sample_combined),
            slide   = as.character(Sample),
            x       = as.numeric(x_centroid),
            y       = as.numeric(y_centroid),
            lineage = as.character(Lineage),
            fine    = as.character(CT_final),
            niche   = as.character(CNiches),
            GA_weeks  = as.numeric(as.character(Gestational_age_weeks)),
            PNA_weeks = as.numeric(as.character(Life_span_weeks))) %>%
  filter(donor %in% COHORT_DONORS) %>%
  mutate(PMA_weeks = GA_weeks + PNA_weeks,
         role = role_of(donor),
         pop  = broad_population(fine),
         mac  = mac_subtype(fine))

donors <- meta %>% distinct(donor, role, GA_weeks, PNA_weeks, PMA_weeks) %>% arrange(role, PMA_weeks)
stopifnot(nrow(donors) == length(COHORT_DONORS),
          all(donors$PNA_weeks[donors$role == ref_role] >= 0))
pma <- with(donors, setNames(PMA_weeks, donor))

imm <- filter(meta, lineage == "Immune")
six <- filter(imm, !is.na(pop))
mac <- filter(imm, !is.na(mac))

cat("\n=== TABLE 1.1  Donors ===\n"); print(donors, n = Inf)
cat("Immune cells:", nrow(imm), "| in six broad populations:", nrow(six),
    "| macrophages:", nrow(mac), "\n")
```

1B. DONOR-LEVEL PSEUDOBULKS

```
counts <- Seurat::GetAssayData(df, assay = "RNA", layer = "counts")

pb_global <- pseudobulk(counts[, imm$cell], imm$donor)
pb_six    <- pseudobulk(counts[, six$cell], paste(six$donor, six$pop, sep = "___"))
pb_mac    <- pseudobulk(counts[, mac$cell], paste(mac$donor, mac$mac, sep = "___"))

n_cells <- bind_rows(count(six, donor, axis = pop, name = "n_cells"),
                     count(mac, donor, axis = mac, name = "n_cells"))
```

1C. LINEAGE SPECIFICITY OF PANEL GENES (spillover-filtered panel)

```
# A gene is predominantly non-immune when a non-immune lineage expresses it at
# >= 5x the immune level (immune CPM / max non-immune CPM < NONIMMUNE_RATIO).

lin_meta <- filter(meta, !is.na(niche), lineage %in% lin_levels)
lin_cpm  <- pseudobulk(counts[, lin_meta$cell], factor(lin_meta$lineage, levels = lin_levels))
lin_cpm  <- t(t(lin_cpm) / colSums(lin_cpm)) * 1e6

gene_lineage <- tibble(
  gene = rownames(lin_cpm),
  immune_ratio = lin_cpm[, "Immune"] /
    pmax(apply(lin_cpm[, setdiff(lin_levels, "Immune"), drop = FALSE], 1, max), 1e-9),
  top_lineage = colnames(lin_cpm)[apply(lin_cpm, 1, which.max)]
) %>% mutate(nonimmune = immune_ratio < NONIMMUNE_RATIO)

nonimmune_genes <- gene_lineage$gene[gene_lineage$nonimmune]
keep_genes      <- gene_lineage$gene[!gene_lineage$nonimmune]

cat("\n=== TABLE 1.2  Predominantly non-immune panel genes:", length(nonimmune_genes),
    "of", nrow(gene_lineage), "===\n")
print(table(gene_lineage$top_lineage[gene_lineage$nonimmune]))
```

1E. SPILLOVER: NON-IMMUNE TRANSCRIPTS IN EACH IMMUNE CELL

```
imm_total  <- Matrix::colSums(counts[, imm$cell, drop = FALSE])
spill_sets <- split(nonimmune_genes, gene_lineage$top_lineage[gene_lineage$nonimmune])
spill <- imap_dfr(spill_sets, \(g, s) tibble(
  cell = imm$cell, gene_set = paste(s, "genes"),
  frac = as.numeric(Matrix::colSums(counts[g, imm$cell, drop = FALSE])) / pmax(imm_total, 1)))
```

3A. REFERENCE-ONLY DEVELOPMENTAL AXES

```
# PCA (centred, scaled) on the 9 reference donors only. The PC of PC1-PC5 most
# associated with PMA is selected; its P value comes from permuting PMA and
# repeating the selection (selection-aware). Every donor is then projected onto
# that PC, oriented so that higher = more mature.

ref_axis_gen <- function(cnt, ref_ids, age, n_perm = N_PERM_PC, seed = 42) {
  lc  <- logcpm(cnt)
  ref <- intersect(ref_ids, colnames(lc))
  lr  <- lc[, ref, drop = FALSE]
  lr  <- lr[apply(lr, 1, stats::var) > 0, , drop = FALSE]
  pca <- stats::prcomp(t(lr), center = TRUE, scale. = TRUE)
  S   <- pca$x[, 1:min(5, ncol(pca$x)), drop = FALSE]
  pr  <- age[ref]
  rho <- stats::cor(S, pr, method = "spearman")[, 1]
  k   <- which.max(abs(rho))
  set.seed(seed)
  null <- replicate(n_perm, max(abs(stats::cor(S, sample(pr), method = "spearman"))))
  nominal <- map_dbl(seq_len(ncol(S)), \(j) suppressWarnings(
    stats::cor.test(S[, j], pr, method = "spearman", exact = FALSE)$p.value))
  sgn  <- sign(rho[k])
  proj <- stats::predict(pca, t(lc[rownames(pca$rotation), , drop = FALSE]))[, k]
  list(pc = k, rho = abs(unname(rho[k])),
       p_perm = (1 + sum(null >= abs(rho[k]))) / (1 + n_perm),
       variance = 100 * pca$sdev[k]^2 / sum(pca$sdev^2),
       score = sgn * proj, loadings = sort(sgn * pca$rotation[, k]), n_genes = nrow(lr),
       pc_table = tibble(PC = seq_len(ncol(S)), variance = 100 * pca$sdev[seq_len(ncol(S))]^2 / sum(pca$sdev^2),
                         rho = rho, p = nominal, FDR = stats::p.adjust(nominal, "BH")))
}
```

```
ref_axis <- function(cnt, n_perm = N_PERM_PC, seed = 42) ref_axis_gen(cnt, REF_DONORS, pma, n_perm, seed)

dev_axes <- map(set_names(c("Global", pop_levels)), \(a) {
  cnt <- if (a == "Global") pb_global else counts_for(pb_six, a)
  list(full     = ref_axis(cnt),
       filtered = ref_axis(cnt[intersect(keep_genes, rownames(cnt)), , drop = FALSE]))
})

axis_summary <- imap_dfr(dev_axes, \(a, nm) imap_dfr(a, \(r, panel)
                                                     tibble(axis = nm, panel = panel, PC = r$pc, n_genes = r$n_genes,
                                                            variance = r$variance, rho = r$rho, p_perm = r$p_perm))) %>%
  group_by(is_pop = axis != "Global", panel) %>%
  mutate(FDR_perm = stats::p.adjust(p_perm, "BH")) %>% ungroup() %>% select(-is_pop)

cat("\n=== TABLE 3.1  Reference-only developmental axes (selection-aware permutation P) ===\n")
print(axis_summary %>% mutate(across(c(variance, rho), \(v) round(v, 3)),
                              across(c(p_perm, FDR_perm), \(v) signif(v, 3))), n = Inf)
cat("\n=== TABLE 3.2  Global axis, PC1-PC5 among reference donors ===\n")
print(dev_axes$Global$full$pc_table %>% mutate(across(where(is.numeric), \(v) signif(v, 3))))
```

3B. DONOR SCORES ON THE REFERENCE AXES

```
make_scored <- function(score) {
  tibble(donor = names(score), Developmental_PC = unname(score)) %>%
    inner_join(donors, by = "donor") %>%
    score_with_loo() %>%
    add_point_style()
}

glob_scored <- make_scored(dev_axes$Global$full$score)
glob_ref    <- filter(glob_scored, role == ref_role)
glob_model  <- fit_reference(glob_ref)
ref_min_pma <- min(glob_ref$PMA_weeks)
ref_max_pma <- max(glob_ref$PMA_weeks)

pop_scored <- map_dfr(pop_levels, \(p)
                      make_scored(dev_axes[[p]]$full$score) %>% mutate(Broad_population = p)) %>%
  left_join(rename(n_cells, Broad_population = axis), by = c("donor", "Broad_population")) %>%
  mutate(n_cells = coalesce(n_cells, 0L), low_cells = n_cells < MIN_CELLS,
         Broad_population = factor(Broad_population, levels = pop_levels))
```

4A. ANALYSIS COHORTS

```
cohort_md <- function(injured) donors %>%
  filter(donor %in% c(REF_DONORS, injured)) %>%
  transmute(donor, PMA = PMA_weeks,
            injury = factor(ifelse(donor %in% injured, "Injured", "Reference"),
                            levels = c("Reference", "Injured"))) %>%
  mutate(PMA_scaled = as.numeric(scale(PMA))) %>%
  arrange(donor) %>% as.data.frame()

md_primary     <- cohort_md(INJURED_DONORS)
md_sensitivity <- cohort_md(SENS_INJURED_DONORS)
```

4B. PERMANOVA: GLOBAL EXPRESSION AND SIX-POPULATION COMPOSITION

```
# Terms entered sequentially (PMA, then injury). The donor order affects the
# permutation stream; ordering by PMA reproduces the reported expression P.
```

```
run_adonis <- function(dist, md) { set.seed(42)
  vegan::adonis2(dist ~ PMA + injury, data = md, permutations = N_PERM_ADON, by = "terms") }
run_dispersion <- function(dist, md) { set.seed(42)
  vegan::permutest(vegan::betadisper(dist, md$injury), permutations = N_PERM_ADON)$tab[1, "Pr(>F)"] }

md_expr   <- md_primary[order(md_primary$PMA, md_primary$donor), ]
lc_global <- logcpm(pb_global)
perm_expr <- run_adonis(stats::dist(t(lc_global[, md_expr$donor])), md_expr)

clr_dist <- function(md) {
  comp <- n_cells %>% filter(axis %in% pop_levels, donor %in% md$donor) %>%
    pivot_wider(names_from = axis, values_from = n_cells, values_fill = 0) %>%
    column_to_rownames("donor")
  comp <- as.matrix(comp[md$donor, pop_levels]) + 0.5
  stats::dist(t(apply(comp / rowSums(comp), 1, \(v) log(v) - mean(log(v)))))
}
perm_comp <- map(list(primary = md_primary, sensitivity = md_sensitivity), \(md) {
  dd <- clr_dist(md)
  list(permanova = run_adonis(dd, md), dispersion_p = run_dispersion(dd, md))
})

tidy_adonis <- function(a, label) as.data.frame(a) %>% rownames_to_column("term") %>%
  mutate(analysis = label, .before = 1)
permanova_tbl <- bind_rows(
  tidy_adonis(perm_expr, "Global expression (primary)"),
  tidy_adonis(perm_comp$primary$permanova, "Composition (primary)"),
  tidy_adonis(perm_comp$sensitivity$permanova, "Composition (+PDL016)"))

cat("\n=== TABLE 4.1  PERMANOVA (adonis2, sequential terms) ===\n")
print(permanova_tbl %>% mutate(across(where(is.numeric), \(v) round(v, 3))))
cat("Composition dispersion P: primary", perm_comp$primary$dispersion_p,
    "| +PDL016", perm_comp$sensitivity$dispersion_p, "\n")
```

4C. DESEQ2: DONOR PSEUDOBULK ~ PMA + INJURY

```
run_deseq <- function(md) {
  cnt <- pb_global[, md$donor]; storage.mode(cnt) <- "integer"
  cd  <- md %>% column_to_rownames("donor") %>% select(PMA_scaled, injury)
  dds <- DESeq2::DESeqDataSetFromMatrix(cnt, cd, design = ~ PMA_scaled + injury)
  dds <- DESeq2::DESeq(dds, quiet = TRUE)
  DESeq2::results(dds, contrast = c("injury", "Injured", "Reference"), alpha = 0.05) %>%
    as.data.frame() %>% rownames_to_column("gene") %>% arrange(pvalue)
}
de <- map(list(primary = md_primary, sensitivity = md_sensitivity), run_deseq)
de_conc <- inner_join(de$primary, de$sensitivity, by = "gene", suffix = c("_primary", "_sens"))
de_rho  <- stats::cor(de_conc$log2FoldChange_primary, de_conc$log2FoldChange_sens, method = "spearman")

cat("\n=== TABLE 4.2  Differential expression (primary: 9 reference vs 4 injured) ===\n")
cat("FDR < 0.05:", sum(de$primary$padj < 0.05, na.rm = TRUE),
    "| nominal P < 0.05:", sum(de$primary$pvalue < 0.05, na.rm = TRUE),
    "| +PDL016 FDR < 0.05:", sum(de$sensitivity$padj < 0.05, na.rm = TRUE),
    "| log2FC concordance rho:", round(de_rho, 3), "\n")
print(head(de$primary %>% mutate(across(where(is.numeric), \(v) signif(v, 3))), 15))
```

5A. GENE-WISE REFERENCE TRAJECTORIES AND OFF-PATH RESIDUALS

```
# For each gene: expression ~ PMA in reference donors. Each donor's deviation is
# the prediction-standardized residual (reference donors leave-one-out).
# Direction test: mean pairwise cosine among injured residual vectors. Exact
# permutation with refitting: for every same-size set S drawn from the pooled
# reference + injured donors, trajectories are refitted to the donors outside S
# and S is scored out of sample, so the observed injured set and every null set
# are treated identically. (Scoring injured donors from one shared fit and
# reference donors leave-one-out is NOT exchangeable: shared-fit residuals are
```

```
# positively correlated, leave-one-out residuals negatively.)

path_residuals <- function(lc, ref) {
  resid_for <- function(train, test) {
    X    <- cbind(1, pma[train])
    XtXi <- solve(crossprod(X))
    Y    <- lc[, train, drop = FALSE]
    B    <- Y %*% X %*% XtXi
    sig  <- sqrt(rowSums((Y - B %*% t(X))^2) / (length(train) - 2))
    x0   <- cbind(1, pma[test])
    h    <- rowSums((x0 %*% XtXi) * x0)
    E    <- lc[, test, drop = FALSE] - B %*% t(x0)
    sweep(E / sig, 2, sqrt(1 + h), "/")
  }
  others <- setdiff(colnames(lc), ref)
  R <- cbind(resid_for(ref, others), sapply(ref, \(i) resid_for(setdiff(ref, i), i)))
  colnames(R) <- c(others, ref)
  R[, colnames(lc), drop = FALSE]
}
cos_mat   <- function(R) { U <- sweep(R, 2, sqrt(colSums(R^2)), "/"); crossprod(U) }
coherence <- function(C, s) { m <- C[s, s, drop = FALSE]; mean(m[upper.tri(m)]) }
resid_out <- function(lc, train, test) {               # out-of-sample residuals for `test`
  X <- cbind(1, pma[train]); XtXi <- solve(crossprod(X))
  Y <- lc[, train, drop = FALSE]; B <- Y %*% X %*% XtXi
  sig <- sqrt(rowSums((Y - B %*% t(X))^2) / (length(train) - 2))
  x0 <- cbind(1, pma[test]); h <- rowSums((x0 %*% XtXi) * x0)
  sweep((lc[, test, drop = FALSE] - B %*% t(x0)) / sig, 2, sqrt(1 + h), "/")
}
coh_of <- function(R) {
  R <- R[apply(is.finite(R), 1, all), , drop = FALSE]
  U <- sweep(R, 2, sqrt(colSums(R^2)), "/"); m <- crossprod(U); mean(m[upper.tri(m)])
}
# Refit permutation test. Gene filter uses all pooled donors (label-free).
refit_test <- function(a, group, filtered, drop_genes = character(0), exclude = character(0)) {
  x <- a$x[, setdiff(intersect(colnames(a$x), a$ok), exclude), drop = FALSE]
  if (filtered) x <- x[intersect(keep_genes, rownames(x)), , drop = FALSE]
  lc <- logcpm(x)
  g <- intersect(group, colnames(lc)); if (length(g) < 3) return(NULL)
  pool <- intersect(c(REF_DONORS, g), colnames(lc))
  lc <- lc[rowMeans(lc[, pool, drop = FALSE]) >= MIN_LOGCPM & !rownames(lc) %in% drop_genes, pool, drop = FALSE]
  stat <- function(S) coh_of(resid_out(lc, setdiff(pool, S), S))
  obs <- stat(g); null <- utils::combn(pool, length(g), stat)
  list(summary = tibble(n_pool = length(pool), n_group = length(g), n_genes = nrow(lc),
                        coherence = obs, null_mean = mean(null),
                        p = mean(null >= obs), n_perm = length(null)),
       null = null)
}
run_offpath <- function(x, ok) {
  x   <- x[, intersect(colnames(x), ok), drop = FALSE]
  lc  <- logcpm(x)
  ref <- intersect(REF_DONORS, colnames(lc))
  lc  <- lc[rowMeans(lc[, ref, drop = FALSE]) >= MIN_LOGCPM, , drop = FALSE]
  R   <- path_residuals(lc, ref)
  R   <- R[apply(is.finite(R), 1, all), , drop = FALSE]
  list(lc = lc, R = R, C = cos_mat(R), ref = ref)
}

op_axes <- map(set_names(axis_levels), \(a) {
  if (a == "Global") return(list(x = pb_global, ok = colnames(pb_global)))
  pb <- if (a %in% mac_levels) pb_mac else pb_six
  list(x = counts_for(pb, a), ok = filter(n_cells, axis == a, n_cells >= MIN_CELLS)$donor)
})
op <- map(op_axes, \(a) list(
  full     = run_offpath(a$x, a$ok),
  filtered = run_offpath(a$x[intersect(keep_genes, rownames(a$x)), , drop = FALSE], a$ok)))

OP_SETS <- list(injured = INJURED_DONORS,
                injured_no_PDL008 = setdiff(INJURED_DONORS, "PDL008"),
                injured_plus_PDL016 = SENS_INJURED_DONORS)

coh_runs <- imap(op_axes, \(a, ax) imap(c(full = FALSE, filtered = TRUE), \(f, panel)
                                        imap(OP_SETS, \(st, set) refit_test(a, st, f))))
coh <- imap_dfr(coh_runs, \(r3, ax) imap_dfr(r3, \(r2, panel) imap_dfr(r2, \(r, set)
                                                                       if (is.null(r)) NULL else mutate(r$summary, axis =
ax, panel = panel, set = set, .before = 1)))) %>%
```

```
  group_by(panel, set) %>% mutate(FDR = stats::p.adjust(p, "BH")) %>% ungroup() %>%
  mutate(axis = factor(axis, levels = axis_levels))
```

6B. DONORS, PSEUDOBULKS AND GENE SET

```
lm_ref_role     <- "LungMAP control"
lm_stage_levels <- c("Control", "aeBPD", "cBPD", "hBPD")
lm_stage_name   <- c(Control = "Term-born control", aeBPD = "Active/evolving BPD",
                     cBPD = "Chronic BPD", hBPD = "Healed BPD")
lm_stage_col    <- c(Control = "#4D4D4D", aeBPD = "#E69F00", cBPD = "#D55E00", hBPD = "#8E3B12")
lm_stage_shape  <- c(Control = 21, aeBPD = 24, cBPD = 24, hBPD = 24)
lm_bpd_col      <- "#A63A0A"                              # BPD as one group (Figure 5)

lm_donors <- lm_raw$obs_donor %>%
  transmute(donor = as.character(donor),
            stage = factor(recode(as.character(disease), control = "Control", eBPD = "cBPD"),
                           levels = lm_stage_levels),
            GA_weeks = as.numeric(as.character(ga)), PMA_weeks = as.numeric(as.character(pma)),
            PNA_weeks = PMA_weeks - GA_weeks, sex = as.character(sex)) %>%
  mutate(role   = if_else(stage == "Control", lm_ref_role, "BPD"),
         injury = factor(if_else(stage == "Control", "Reference", "Injured"), levels = c("Reference", "Injured"))) %>%
  arrange(stage, PMA_weeks)
stopifnot(!anyNA(lm_donors$stage), !anyDuplicated(lm_donors$donor))
lm_pma  <- with(lm_donors, setNames(PMA_weeks, donor))
lm_ctrl <- lm_donors$donor[lm_donors$role == lm_ref_role]

lm_pb  <- lm_raw$pb
lm_lab <- sub(".*___", "", colnames(lm_pb))
lm_don <- sub("___.*", "", colnames(lm_pb))
lm_pop <- unname(lm_label_pop[lm_lab])
in_six <- lm_pop != "Other"
lm_pb_six    <- pseudobulk(lm_pb[, in_six], paste(lm_don, lm_pop, sep = "___")[in_six])
lm_pb_global <- pseudobulk(lm_pb[, in_six], lm_don[in_six])

lm_n_cells <- lm_raw$n_labelled %>%
  filter(label %in% names(lm_label_pop)) %>%
  mutate(axis = unname(lm_label_pop[label])) %>%
  group_by(donor, axis) %>% summarise(n_cells = sum(n), .groups = "drop") %>%
  complete(donor = lm_donors$donor, axis = comp_levels, fill = list(n_cells = 0L))

lm_panel <- intersect(rownames(pb_global), rownames(lm_pb))
cat("\n=== TABLE 6.1  LungMAP donors (immune nuclei per compartment) ===\n")
cat("Xenium panel genes found in LungMAP:", length(lm_panel), "of", nrow(pb_global), "\n")
print(lm_donors %>% left_join(lm_n_cells %>% pivot_wider(names_from = axis, values_from = n_cells), by = "donor") %>%
        select(donor, stage, GA_weeks, PMA_weeks, sex, all_of(comp_levels)), n = Inf, width = Inf)
```

6C. REFERENCE-ONLY AXES (LungMAP CONTROLS) AND DEVIATIONS

```
lm_ok <- function(p) lm_n_cells %>% filter(axis == p, n_cells >= MIN_CELLS) %>% pull(donor)
lm_axes <- map(set_names(c("Global", pop_levels)), \(a) {
  cnt <- if (a == "Global") lm_pb_global else counts_for(lm_pb_six, a)
  cnt <- cnt[lm_panel, intersect(colnames(cnt), if (a == "Global") lm_donors$donor else lm_ok(a)), drop = FALSE]
  if (sum(colnames(cnt) %in% lm_ctrl) < 6 || sum(!colnames(cnt) %in% lm_ctrl) < 3) return(NULL)
  ref_axis_gen(cnt, lm_ctrl, lm_pma)
}) %>% compact()

lm_axis_summary <- imap_dfr(lm_axes, \(r, nm) tibble(axis = nm, PC = r$pc, n_genes = r$n_genes,
                                                     variance = r$variance, rho = r$rho, p_perm = r$p_perm)) %>%
  mutate(FDR_perm = stats::p.adjust(p_perm, "BH"))

make_scored_lm <- function(score) tibble(donor = names(score), Developmental_PC = unname(score)) %>%
  inner_join(lm_donors, by = "donor") %>% score_with_loo(ref = lm_ref_role)
```

```
lm_glob_scored <- make_scored_lm(lm_axes$Global$score)
lm_pop_scored  <- imap_dfr(lm_axes[names(lm_axes) != "Global"], \(r, p)
                           make_scored_lm(r$score) %>% mutate(Broad_population = p)) %>%
  left_join(rename(lm_n_cells, Broad_population = axis), by = c("donor", "Broad_population")) %>%
  mutate(low_cells = n_cells < MIN_CELLS)

lm_comp_scored <- lm_n_cells %>%
  group_by(donor) %>% mutate(total = sum(n_cells), share = n_cells / total) %>% ungroup() %>%
  rename(compartment = axis, n = n_cells) %>%
  mutate(Developmental_PC = stats::qlogis((n + 0.5) / (total + 1))) %>%
  inner_join(lm_donors, by = "donor") %>%
  group_by(compartment) %>% group_modify(\(x, ...) score_with_loo(x, ref = lm_ref_role)) %>% ungroup()

cat("\n=== TABLE 6.2  LungMAP reference-only axes (controls; selection-aware P) ===\n")
print(lm_axis_summary %>% mutate(across(where(is.double), \(v) signif(v, 3))), n = Inf)
cat("\n=== TABLE 6.3  BPD donors beyond the 95% limit (maturation and composition) ===\n")
print(bind_rows(lm_glob_scored %>% mutate(axis = "Global"),
                lm_pop_scored %>% filter(!low_cells) %>% mutate(axis = Broad_population),
                lm_comp_scored %>% mutate(axis = paste("Share:", compartment))) %>%
        filter(role != lm_ref_role, exceeds) %>%
        select(axis, donor, stage, PMA_weeks, dev_score, limit, extrapolated) %>%
        mutate(across(c(dev_score, limit), \(v) round(v, 2))), n = Inf)
```

7A. PERMANOVA: AGE VS INJURY (MARGINAL = PRIMARY; SEQUENTIAL AS IN XENIUM)

```
r2_margin <- function(Y, age, inj) {                       # Euclidean = multivariate LS
  Y <- scale(Y, scale = FALSE); tss <- sum(Y^2)
  rss <- function(X) sum(qr.resid(qr(X), Y)^2)
  rf  <- rss(cbind(1, age, inj))
  c(PMA = (rss(cbind(1, inj)) - rf) / tss, injury = (rss(cbind(1, age)) - rf) / tss)
}
null_r2 <- function(Y, md, n_perm = 999, seed = 42) {
  set.seed(seed); inj <- md$injury == "Injured"
  b <- t(replicate(n_perm, r2_margin(Y[sample(nrow(Y)), , drop = FALSE], md$PMA, inj)))
  tibble(term = c("PMA", "injury"), null_R2 = colMeans(b))
}
permanova_cohort <- function(Y, md, cohort, analysis) {   # Y: donors x features, rows = md$donor
  dd <- stats::dist(Y)
  tid <- function(a, type) as.data.frame(a) %>% rownames_to_column("term") %>% as_tibble() %>%
    filter(term %in% c("PMA", "injury")) %>% transmute(term, type, R2, F, P = `Pr(>F)`)
  set.seed(42); sq <- vegan::adonis2(dd ~ PMA + injury, data = md, permutations = N_PERM_ADON, by = "terms")
  set.seed(42); mg <- vegan::adonis2(dd ~ PMA + injury, data = md, permutations = N_PERM_ADON, by = "margin")
  bind_rows(tid(mg, "marginal"), tid(sq, "sequential")) %>%
    left_join(null_r2(Y, md), by = "term") %>%
    mutate(excess_R2 = R2 - null_R2, cohort = cohort, analysis = analysis,
           n_ref = sum(md$injury == "Reference"), n_inj = sum(md$injury == "Injured"), .before = 1)
}
clr_rows <- function(comp) { comp <- comp + 0.5; t(apply(comp / rowSums(comp), 1, \(v) log(v) - mean(log(v)))) }

# Xenium (9 reference vs 4 injured), same inputs as Table 4.1
xen_comp <- n_cells %>% filter(axis %in% pop_levels, donor %in% md_primary$donor) %>%
  pivot_wider(names_from = axis, values_from = n_cells, values_fill = 0) %>%
  column_to_rownames("donor")
xen_comp <- as.matrix(xen_comp[md_primary$donor, pop_levels])

# LungMAP (11 controls vs 13 BPD)
md_lm <- lm_donors %>% transmute(donor, PMA = PMA_weeks, injury) %>% arrange(PMA, donor) %>% as.data.frame()
lc_lm_panel <- logcpm(lm_pb_global[lm_panel, md_lm$donor])
lc_lm_panel <- lc_lm_panel[apply(lc_lm_panel, 1, stats::var) > 0, ]
lc_lm_all   <- logcpm(lm_pb_global[, md_lm$donor])
lm_ctrl_ord <- md_lm$donor[md_lm$injury == "Reference"]
lm_expressed <- rownames(lc_lm_all)[rowMeans(lc_lm_all[, lm_ctrl_ord]) > 1]
lm_hvg <- names(sort(apply(lc_lm_all[lm_expressed, lm_ctrl_ord], 1, stats::var), decreasing = TRUE)) %>% head(2000)
lc_lm_hvg <- logcpm(lm_pb_global[lm_hvg, md_lm$donor])
lm_comp <- lm_n_cells %>% filter(axis %in% pop_levels) %>%
  pivot_wider(names_from = axis, values_from = n_cells, values_fill = 0) %>% column_to_rownames("donor")
lm_comp <- as.matrix(lm_comp[md_lm$donor, pop_levels])

xcohort <- "Xenium: preterm injury"
```

```
lcohort <- "LungMAP: BPD"
age_injury_tbl <- bind_rows(
  permanova_cohort(t(lc_global[, md_expr$donor]), md_expr, xcohort, "Expression"),
  permanova_cohort(clr_rows(xen_comp), md_primary, xcohort, "Composition"),
  permanova_cohort(t(lc_lm_panel), md_lm, lcohort, "Expression"),
  permanova_cohort(clr_rows(lm_comp), md_lm, lcohort, "Composition"),
  permanova_cohort(t(lc_lm_hvg), md_lm, lcohort, "Expression (2,000 HVG; sensitivity)"))

cat("\n=== TABLE 7.1  Age vs injury: marginal PERMANOVA R2 (primary) with permutation-null R2 ===\n")
print(age_injury_tbl %>% filter(type == "marginal") %>%
        select(cohort, analysis, n_ref, n_inj, term, R2, null_R2, excess_R2, F, P) %>%
        mutate(across(c(R2, null_R2, excess_R2), \(v) round(v, 3)), F = round(F, 2), P = signif(P, 3)), n = Inf)
cat("\n=== TABLE 7.2  Sequential terms (PMA entered first) ===\n")
print(age_injury_tbl %>% filter(type == "sequential") %>% select(cohort, analysis, term, R2, F, P) %>%
        mutate(across(c(R2, F), \(v) round(v, 3)), P = signif(P, 3)), n = Inf)
```

7B. INJURY SHIFT ON EVERY AXIS, WITH THE SHIFT THE DESIGN COULD DETECT

```
# Model per axis: score ~ PMA + injured (reference + injured donors). Shift and
# CI are in units of the model residual SD; MDE80 = shift detected with 80%
# power (two-sided alpha 0.05, noncentral t, this design).

injury_shift <- function(d) {
  d <- filter(d, !is.na(Developmental_PC))
  if (sum(d$injured) < 3 || sum(!d$injured) < 6) return(NULL)
  s  <- summary(stats::lm(Developmental_PC ~ PMA_weeks + injured, data = d))
  co <- s$coefficients["injuredTRUE", ]; sig <- s$sigma; df <- s$df[2]
  tc <- stats::qt(0.975, df); v <- (co[["Std. Error"]] / sig)^2
  pow <- function(delta) { ncp <- delta / sqrt(v); stats::pt(-tc, df, ncp) + 1 - stats::pt(tc, df, ncp) }
  tibble(n_ref = sum(!d$injured), n_inj = sum(d$injured),
         shift_SD = co[["Estimate"]] / sig,
         lo = (co[["Estimate"]] - tc * co[["Std. Error"]]) / sig,
         hi = (co[["Estimate"]] + tc * co[["Std. Error"]]) / sig,
         p = co[["Pr(>|t|)"]],
         MDE80_SD = stats::uniroot(\(x) pow(x) - 0.8, c(0, 100))$root)
}

shift_input <- bind_rows(
  glob_scored %>% mutate(axis = "Global", measure = "Maturation"),
  pop_scored %>% filter(!low_cells) %>% mutate(axis = as.character(Broad_population), measure = "Maturation"),
  comp_scored %>% mutate(axis = compartment, measure = "Composition")) %>%
  filter(role %in% c(ref_role, inj_role)) %>%
  transmute(cohort = xcohort, measure, axis, donor, PMA_weeks, Developmental_PC, injured = role == inj_role) %>%
  bind_rows(
    bind_rows(lm_glob_scored %>% mutate(axis = "Global", measure = "Maturation"),
              lm_pop_scored %>% filter(!low_cells) %>% mutate(axis = Broad_population, measure = "Maturation"),
              lm_comp_scored %>% mutate(axis = compartment, measure = "Composition")) %>%
      transmute(cohort = lcohort, measure, axis, donor, PMA_weeks, Developmental_PC, injured = role != lm_ref_role))

shift_tbl <- shift_input %>%
  group_by(cohort, measure, axis) %>% group_modify(\(x, ...) { r <- injury_shift(x); if (is.null(r)) tibble() else r })
%>% ungroup() %>%
  group_by(cohort, measure) %>% mutate(FDR = stats::p.adjust(p, "BH")) %>% ungroup() %>%
  mutate(measure = factor(measure, levels = c("Maturation", "Composition")),
         axis = factor(axis, levels = c("Global", comp_levels)),
         cohort = factor(cohort, levels = c(xcohort, lcohort))) %>%
  arrange(measure, axis, cohort)

cat("\n=== TABLE 7.3  Injury shift per axis (residual SD), 95% CI, and detectable shift (MDE80) ===\n")
print(shift_tbl %>% mutate(across(c(shift_SD, lo, hi, MDE80_SD), \(v) round(v, 2)),
                           across(c(p, FDR), \(v) signif(v, 3))), n = Inf, width = Inf)
```

7C. FROZEN ALVEOLAR-MACROPHAGE SIGNATURE IN LungMAP (pre-specified)

```
# Signature fixed from the Xenium off-path result before LungMAP was opened:
```

```
# genes with the same sign in all injured donors, |mean deviation| >= 0.5 and
# immune-specific (immune ratio >= 1). Score per donor: alveolar-macrophage
# pseudobulk (>= 20 nuclei), z across donors, mean(up) - mean(down);
# score ~ BPD + PMA, one-sided (BPD > control).

am_signature <- am_genes %>%
  filter(n_same_sign == length(inj_am), abs(mean_z) >= 0.5, immune_ratio >= 1) %>%
  transmute(gene, direction = if_else(mean_z > 0, "up", "down"), weight = mean_z, module)

lm_am_n  <- lm_raw$n_labelled %>% filter(label == "AM") %>% select(donor, n_am = n)
lm_pb_am <- lm_pb[, lm_lab == "AM", drop = FALSE]; colnames(lm_pb_am) <- lm_don[lm_lab == "AM"]
lm_pb_am <- lm_pb_am[, intersect(colnames(lm_pb_am), filter(lm_am_n, n_am >= 20)$donor), drop = FALSE]
sig_lm   <- filter(am_signature, gene %in% rownames(lm_pb_am))
z_am     <- t(scale(t(logcpm(lm_pb_am)[sig_lm$gene, , drop = FALSE]))); z_am[!is.finite(z_am)] <- 0
up_g <- sig_lm$gene[sig_lm$direction == "up"]; dn_g <- sig_lm$gene[sig_lm$direction == "down"]

lm_signature_donors <- tibble(donor = colnames(z_am),
                              score = colMeans(z_am[up_g, , drop = FALSE]) - colMeans(z_am[dn_g, , drop = FALSE]),
                              weighted = as.numeric(crossprod(sig_lm$weight, z_am)) / sum(abs(sig_lm$weight))) %>%
  left_join(lm_am_n, by = "donor") %>% left_join(lm_donors, by = "donor")
sig_test <- function(y) {
  s <- summary(stats::lm(stats::reformulate(c("injury", "PMA_weeks"), y), data = lm_signature_donors))
  co <- s$coefficients["injuryInjured", ]
  tibble(endpoint = y, estimate = co[["Estimate"]], se = co[["Std. Error"]],
         p_one_sided = stats::pt(co[["t value"]], df = s$df[2], lower.tail = FALSE))
}
lm_signature_test <- bind_rows(sig_test("score") %>% mutate(type = "Primary"),
                               sig_test("weighted") %>% mutate(type = "Secondary")) %>%
  mutate(n_genes = nrow(sig_lm), n_signature = nrow(am_signature),
         n_ctrl = sum(lm_signature_donors$injury == "Reference"), n_bpd = sum(lm_signature_donors$injury == "Injured"))

cat("\n=== TABLE 7.4  Frozen AM signature in LungMAP (BPD vs control, PMA-adjusted; one-sided) ===\n")
cat("Signature genes:", nrow(am_signature), "| found in LungMAP:", nrow(sig_lm), "\n")
print(lm_signature_test %>% mutate(across(c(estimate, se, p_one_sided), \(v) round(v, 3))))
```

7L. DONOR-SPECIFIC OFF-PATH MAGNITUDE (XENIUM)

```
# The coherence test (Figure S3) asks whether injured donors leave the path in
# the SAME direction. Injury could instead push each donor off the path in its
# own direction. Statistic: mean, over the injured set, of each donor's
# root-mean-square prediction-standardized residual (any direction). Exact
# refit permutation over all 4-donor sets of the 13 pooled donors, exactly as
# for coherence. Per donor: magnitude when scored from a fit to the other 12
# donors (symmetric for every donor), ranked against reference donors.

rms_cols <- function(R) { R <- R[apply(is.finite(R), 1, all), , drop = FALSE]; sqrt(colMeans(R^2)) }

mag_runs <- compact(imap(op_axes, \(a, ax) {
  x <- a$x[, intersect(colnames(a$x), a$ok), drop = FALSE]
  x <- x[intersect(keep_genes, rownames(x)), , drop = FALSE]
  lc <- logcpm(x)
  g <- intersect(INJURED_DONORS, colnames(lc)); if (length(g) < 3) return(NULL)
  pool <- intersect(c(REF_DONORS, g), colnames(lc))
  lc <- lc[rowMeans(lc[, pool, drop = FALSE]) >= MIN_LOGCPM, pool, drop = FALSE]
  stat <- function(S) mean(rms_cols(resid_out(lc, setdiff(pool, S), S)))
  obs  <- stat(g); null <- utils::combn(pool, length(g), stat)
  per_donor <- map_dbl(set_names(pool), \(i) rms_cols(resid_out(lc, setdiff(pool, i), i)))
  list(summary = tibble(axis = ax, n_pool = length(pool), n_inj = length(g), n_genes = nrow(lc),
                        magnitude = obs, null_mean = mean(null), p = mean(null >= obs), n_perm = length(null)),
       null = null,
       donors = tibble(axis = ax, donor = names(per_donor), rms = unname(per_donor)) %>%
         mutate(injured = donor %in% g, pct_rank = map_dbl(rms, \(v) mean(rms[!injured] < v))))
}))
mag_tbl <- map_dfr(mag_runs, "summary") %>%
  mutate(FDR = stats::p.adjust(p, "BH"), axis = factor(axis, levels = axis_levels)) %>% arrange(axis)
mag_donors <- map_dfr(mag_runs, "donors") %>% left_join(select(donors, donor, PMA_weeks, PNA_weeks), by = "donor")

cat("\n=== TABLE 7.7  Off-path magnitude of injured donors (exact refit permutation, one-sided) ===\n")
print(mag_tbl %>% mutate(across(c(magnitude, null_mean), \(v) round(v, 3)), across(c(p, FDR), \(v) signif(v, 3))), n =
Inf)
```

```
cat("\n=== TABLE 7.8  Per-donor off-path magnitude (pct_rank = share of reference donors exceeded) ===\n")
print(mag_donors %>% filter(injured) %>% select(axis, donor, PMA_weeks, PNA_weeks, rms, pct_rank) %>%
        mutate(rms = round(rms, 2), pct_rank = round(pct_rank, 2)) %>% arrange(axis, donor), n = Inf)

null_mag <- imap_dfr(mag_runs, \(r, a) tibble(axis = a, null = r$null)) %>%
  mutate(axis = factor(axis, levels = rev(axis_levels)))
figure3d_mag <- ggplot(null_mag, aes(null, axis)) +
  geom_violin(fill = "#E3E3E3", color = NA, scale = "width", width = 0.75) +
  geom_point(data = mag_donors %>% filter(injured) %>% mutate(axis = factor(axis, levels = levels(null_mag$axis))),
             aes(rms, axis), shape = 24, size = 1.1, fill = role_colors[[inj_role]], color = "white",
             stroke = 0.2, alpha = 0.7, position = position_jitter(height = 0.12, width = 0, seed = 42)) +
  geom_point(data = mag_tbl %>% mutate(axis = factor(as.character(axis), levels = levels(null_mag$axis))),
             aes(magnitude, axis), shape = 24, size = 2.4, fill = role_colors[[inj_role]], color = "#333333", stroke =
0.4) +
  geom_text(data = mag_tbl %>% mutate(axis = factor(as.character(axis), levels = levels(null_mag$axis))),
            aes(x = Inf, y = axis, label = paste0("P ", format_p(p))), hjust = 1.05,
            size = pt(5.6), family = font_family, color = "#303030") +
  scale_y_discrete(labels = pop_md) +
  scale_x_continuous(expand = expansion(mult = c(0.04, 0.3))) +
  labs(x = "Off-path magnitude (RMS residual, prediction SD)", y = NULL,
       title = "How far injured donors sit off the path, in any direction",
       caption = "Large triangle, injured mean; small, each injured donor;\ngrey, exact refit-permutation null.") +
  theme_fig() + theme(axis.text.y.left = ggtext::element_markdown(size = 6.3), panel.grid.major.y = element_blank(),
                      plot.caption = element_text(size = 5.6, color = "#505050", hjust = 0))


# SECTION 8 - ALVEOLAR-MACROPHAGE COMPARTMENT (SUPPLEMENTARY FIGURE S7 PANELS)----
```

8A. ALVEOLAR SHARE OF MACROPHAGES VS AGE

```
am_share <- mac %>%
  count(donor, mac) %>%
  pivot_wider(names_from = mac, values_from = n, values_fill = 0) %>%
  transmute(donor, n_am = `Alveolar macrophage`, n_mac = n_am + `SPP1+ macrophage`,
            share = (n_am + 0.5) / (n_mac + 1), Developmental_PC = stats::qlogis(share)) %>%
  left_join(donors, by = "donor") %>%
  score_with_loo() %>%
  add_point_style()

cat("\n=== TABLE 8.1  Alveolar share of macrophages vs age expectation ===\n")
print(am_share %>% transmute(donor, role, PNA_weeks, PMA_weeks, n_mac, share = round(share, 3),
                             expected = round(stats::plogis(expected), 3),
                             dev_score = round(dev_score, 2), extrapolated) %>%
        arrange(role, PMA_weeks), n = Inf)

cat("\n=== TABLE 8.2  Cycling alveolar macrophages ===\n")
print(cycling %>% transmute(donor, role, PNA_weeks, PMA_weeks, n_am, n_cycling_am,
                            pct_am = round(pct_am, 2), pct_struct = round(pct_struct, 2)) %>%
        arrange(role, PMA_weeks), n = Inf)
cat("Spearman rho, AM vs structural-cell cycling (donors with >=", MIN_CELLS, "AM):",
    round(with(filter(cycling, n_am >= MIN_CELLS), stats::cor(pct_am, pct_struct, method = "spearman")), 2), "\n")
```

9A. SETTINGS

```
R_MAIN         <- 40                # neighbourhood radius (microns)
R_SENS         <- c(25, 40, 60)     # radius sensitivity
K_MAX          <- 200               # max neighbours returned per AM (checked below)
MIN_STRUCT_NB  <- 3                 # AMs need >= this many structural neighbours
MAP_DONORS_NB  <- c("PDL013", "PDL009")   # 36-wk reference, 31-wk injured

# Source annotation (CT_final) -> neighbour class. First matching pattern wins.
# Check the crosswalk printed in 7B and edit patterns if a type is misassigned.
NB_PATTERNS <- c(
  "Immature AT2"      = "immature",
```

```
  "Transitional AT2"  = "transitional",
  "AT1"               = "^AT1$",
  "AT2"               = "^AT2$",
  "Capillary"         = "capill|^gcap|^acap|cap$",
  "Fibroblast"        = "fibro",
  "Airway epithelium" = "secret|goblet|basal|cilia|club|pnec|neuroendo")
alv_epi <- c("AT1", "AT2", "Immature AT2", "Transitional AT2")

STATE_UP   <- c("CD68", "PPARG", "C1QC", "FCGR3A", "MARCO", "MCEMP1", "MS4A7", "CST3", "TREM2", "MRC1")
STATE_DOWN <- c("SPP1", "CCL2", "S100A8", "S100A9", "CCL22", "LGALS1", "FCN1", "CD14")

feat_levels <- c("AT1", "AT2", "Immature AT2", "Capillary", "Fibroblast",
                 "Alveolar epithelium", "AT1 maturity index")
feat_cols   <- c("f_AT1", "f_AT2", "f_ImmAT2", "f_Cap", "f_Fib", "f_AlvEpi", "maturity")
feat_lab    <- setNames(feat_levels, feat_cols)
feat_colors <- c("AT1" = "#B5654A", "AT2" = "#D98C5F", "Immature AT2" = "#E8B04A",
                 "Capillary" = "#4F74A3", "Fibroblast" = "#6B7F55",
                 "Alveolar epithelium" = "#9C4A2F", "AT1 maturity index" = "#5B2A86")
label_colors <- c(label_colors, feat_colors)

nb_map_colors <- c("AT1" = "#E3A58C", "AT2" = "#F0C4A2", "Immature AT2" = "#F2D58A",
                   "Transitional AT2" = "#EBC9B5", "Capillary" = "#A9C1E0",
                   "Fibroblast" = "#C3D1AD", "Airway epithelium" = "#D9B8C9",
                   "Other structural" = "#E3E3E3")
```

9C. FIXED-RADIUS NEIGHBOR SEARCH (per slide)

```
nb_long <- map_dfr(unique(am_nb$slide), function(s) {
  pool <- filter(meta_nb, slide == s)
  q    <- filter(am_nb, slide == s)
  nn <- RANN::nn2(as.matrix(pool[, c("x", "y")]), as.matrix(q[, c("x", "y")]),
                  k = min(K_MAX, nrow(pool)), searchtype = "radius", radius = max(R_SENS))
  hit <- which(nn$nn.idx > 0, arr.ind = TRUE)
  tibble(cell = q$cell[hit[, 1]], nb = pool$cell[nn$nn.idx[hit]],
         dist = nn$nn.dists[hit], nb_class = pool$nb_class[nn$nn.idx[hit]]) %>%
    filter(nb != cell)
})
n_capped <- nb_long %>% count(cell) %>% filter(n >= K_MAX - 1) %>% nrow()
if (n_capped > 0) warning(n_capped, " AMs hit K_MAX neighbours; raise K_MAX")

nb_features <- function(r) {
  w <- nb_long %>% filter(dist <= r) %>% count(cell, nb_class) %>%
    pivot_wider(names_from = nb_class, values_from = n, values_fill = 0)
  for (k in c(names(NB_PATTERNS), "Other structural", "Immune")) if (!k %in% names(w)) w[[k]] <- 0L
  select(am_nb, cell) %>% left_join(w, by = "cell") %>%
    mutate(across(-cell, \(v) coalesce(as.numeric(v), 0)),
           n_struct = rowSums(across(all_of(c(names(NB_PATTERNS), "Other structural")))),
           n_alv    = rowSums(across(all_of(alv_epi))),
           f_AT1    = AT1 / n_struct,
           f_AT2    = AT2 / n_struct,
           f_ImmAT2 = `Immature AT2` / n_struct,
           f_Cap    = Capillary / n_struct,
           f_Fib    = Fibroblast / n_struct,
           f_AlvEpi = n_alv / n_struct,
           maturity = if_else(n_alv > 0, AT1 / n_alv, NA_real_)) %>%
    mutate(across(all_of(feat_cols), \(v) if_else(n_struct > 0, v, NA_real_))) %>%
    select(cell, n_struct, n_immune = Immune, n_alv, all_of(feat_cols))
}
```

9D. PER-CELL ALVEOLAR-MACROPHAGE STATE SCORE (immune-specific genes)

```
imm_specific <- gene_lineage$gene[gene_lineage$immune_ratio >= 1]
state_up   <- intersect(STATE_UP, imm_specific)
```

```
state_down <- intersect(STATE_DOWN, imm_specific)
epi_spill_genes <- gene_lineage$gene[gene_lineage$nonimmune & gene_lineage$top_lineage == "Epithelial"]
cat("\nState score genes (immune-specific only)\n  up:  ", paste(state_up, collapse = ", "),
    "\n  down:", paste(state_down, collapse = ", "), "\n")

cnt <- Seurat::GetAssayData(GSE297945_final_complete_cniches_tniches_allsamples_2025_05_26,
                            assay = "RNA", layer = "counts")[, am_nb$cell]
lib_imm  <- Matrix::colSums(cnt[intersect(keep_genes, rownames(cnt)), , drop = FALSE])
expr     <- log1p(t(t(as.matrix(cnt[c(state_up, state_down), , drop = FALSE])) / pmax(lib_imm, 1)) * 1e4)
z        <- t(scale(t(expr)))
am_state <- tibble(cell = am_nb$cell,
                   state = colMeans(z[state_up, , drop = FALSE]) - colMeans(z[state_down, , drop = FALSE]),
                   spill_epi = as.numeric(Matrix::colSums(cnt[intersect(epi_spill_genes, rownames(cnt)), , drop =
FALSE])) /
                     pmax(Matrix::colSums(cnt), 1))
rm(cnt, expr, z); invisible(gc())

am_cell <- am_nb %>%
  select(cell, donor, slide, x, y, role, PMA_weeks, PNA_weeks) %>%
  left_join(am_state, by = "cell") %>%
  left_join(nb_features(R_MAIN), by = "cell")
am_use <- filter(am_cell, n_struct >= MIN_STRUCT_NB)

cat("AMs with >=", MIN_STRUCT_NB, "structural neighbours within", R_MAIN, "um:",
    nrow(am_use), "of", nrow(am_cell), "\n")
```

9G. PART 2 - WITHIN-DONOR: DOES STATE TRACK THE NEIGHBORHOOD?

```
partial_rho <- function(a, b, ctrl) {                    # Spearman, partialling ctrl
  ok <- stats::complete.cases(a, b, ctrl)
  ra <- stats::residuals(stats::lm(rank(a[ok]) ~ rank(ctrl[ok])))
  rb <- stats::residuals(stats::lm(rank(b[ok]) ~ rank(ctrl[ok])))
  stats::cor(ra, rb)
}

within_rho <- function(cells) {
  cells %>%
    group_by(donor, role) %>%
    filter(n() >= MIN_CELLS) %>%
    group_modify(\(d, ...) map_dfr(feat_cols, \(f) {
      ok <- !is.na(d[[f]])
      tibble(feature = f, n_cells = sum(ok),
             rho = if (sum(ok) >= 20) suppressWarnings(stats::cor(d$state[ok], d[[f]][ok], method = "spearman")) else
NA_real_,
             rho_partial = if (sum(ok) >= 20) partial_rho(d$state, d[[f]], d$spill_epi) else NA_real_,
             rho_spill = if (sum(ok) >= 20) suppressWarnings(stats::cor(d$spill_epi[ok], d[[f]][ok], method =
"spearman")) else NA_real_)
    })) %>% ungroup()
}

summarise_rho <- function(r, col = "rho") {
  r %>% filter(!is.na(.data[[col]])) %>%
    group_by(feature) %>%
    summarise(n_donors = n(), median_rho = stats::median(.data[[col]]),
              n_positive = sum(.data[[col]] > 0),
              sign_p = stats::binom.test(sum(.data[[col]] > 0), n(), 0.5)$p.value,
              wilcoxon_p = suppressWarnings(stats::wilcox.test(.data[[col]], mu = 0, exact = TRUE)$p.value),
              mean_rho_ref = mean(.data[[col]][role == ref_role]),
              mean_rho_injured = mean(.data[[col]][role == inj_role]), .groups = "drop") %>%
    mutate(feature = factor(feat_lab[feature], levels = feat_levels)) %>% arrange(feature)
}

rho_main <- within_rho(am_use)

cat("\n=== TABLE 9.4  Within-donor Spearman rho, AM state vs neighbourhood (", R_MAIN, "um) ===\n")
print(summarise_rho(rho_main, "rho") %>% mutate(across(where(is.double), \(v) signif(v, 3))))
cat("\n=== TABLE 9.5  Same, partialling epithelial spillover into the AM ===\n")
print(summarise_rho(rho_main, "rho_partial") %>% mutate(across(where(is.double), \(v) signif(v, 3))))
cat("\n=== TABLE 9.6  Spillover check: epithelial transcripts in AMs vs neighbourhood ===\n")
```

```
print(summarise_rho(rho_main, "rho_spill") %>% mutate(across(where(is.double), \(v) signif(v, 3))))

rho_radius <- map_dfr(R_SENS, \(r) {
  cells <- am_cell %>% select(-any_of(c(feat_cols, "n_struct", "n_immune", "n_alv"))) %>%
    left_join(nb_features(r), by = "cell") %>% filter(n_struct >= MIN_STRUCT_NB)
  summarise_rho(within_rho(cells), "rho") %>% mutate(radius_um = r, .before = 1)
})
cat("\n=== TABLE 9.7  Radius sensitivity (within-donor rho) ===\n")
print(rho_radius %>% select(radius_um, feature, n_donors, median_rho, n_positive, sign_p, wilcoxon_p) %>%
        mutate(across(where(is.double), \(v) signif(v, 3))), n = Inf)
```

10A. SETTINGS

```
DC_FINE        <- c("cDC", "pDC")
DC_MARKERS     <- c("CD1C", "FCER1A", "LILRA4", "GZMB")   # as in the exploratory analysis; GZMB is also
# expressed by pDCs AND cytotoxic lymphocytes
PROLIF_LABELS  <- c("Proliferating monocytes", "Proliferating lymphoid")
CYCLE_MIN      <- 2
DC_MIN_N       <- 20          # cells per donor for a cycling fraction
LM_DC_MIN_N    <- 10          # LungMAP DC nuclei per donor (pre-specified)
THIN_UMI       <- c(40, 50, 60)
N_THIN         <- 200         # rarefaction repetitions per depth
DC_R           <- 40          # neighbourhood radius (um)
DC_K_MAX       <- 300
dc_nb_levels   <- c("AT1", "AT2 lineage", "Airway epithelium", "Endothelial", "Mesenchymal",
                    "T/NK", "B cell", "Myeloid")
dc_nb_colors   <- c("AT1" = "#B5654A", "AT2 lineage" = "#D98C5F", "Airway epithelium" = "#C27BA0",
                    "Endothelial" = "#4F74A3", "Mesenchymal" = "#6B7F55", "T/NK" = "#8E5A9E",
                    "B cell" = "#C99A00", "Myeloid" = "#3B6FA1")
label_colors <- c(label_colors, dc_nb_colors, "DC (source)" = "#3F8F45", "DC (expanded)" = "#2E6B33",
                  "cDC" = "#3F8F45", "pDC" = "#9CCB8F", "Other immune" = "#8C8C8C",
                  "Epithelial" = "#B5654A", "Endothelial" = "#4F74A3", "Mesenchymal" = "#6B7F55")
```

10B. PER-CELL CYCLING CALLS (XENIUM)

```
so_dc  <- GSE297945_final_complete_cniches_tniches_allsamples_2025_05_26
cnt_dc <- Seurat::GetAssayData(so_dc, assay = "RNA", layer = "counts")
cyc_g  <- intersect(CYCLE_GENES, rownames(cnt_dc))
dcm_g  <- intersect(DC_MARKERS, rownames(cnt_dc))
cat("\nCycling genes:", paste(cyc_g, collapse = ", "), "| DC markers on panel:", paste(dcm_g, collapse = ", "), "\n")

cells_cc <- filter(meta, !is.na(lineage))
cyc_counts <- cnt_dc[cyc_g, cells_cc$cell, drop = FALSE]
umi_cc <- Matrix::colSums(cnt_dc[, cells_cc$cell, drop = FALSE])     # all panel genes define depth
cell_cyc <- cells_cc %>%
  select(cell, donor, slide, x, y, lineage, fine, pop, niche_class, role, PMA_weeks, PNA_weeks) %>%
  mutate(n_cyc = as.integer(Matrix::colSums(cyc_counts > 0)), umi = as.numeric(umi_cc),
         dc_marker = as.logical(Matrix::colSums(cnt_dc[dcm_g, cell, drop = FALSE] > 0) > 0),
         cycling = n_cyc >= CYCLE_MIN)
dc_cyc_counts <- as.matrix(cyc_counts[, cell_cyc$fine %in% DC_FINE, drop = FALSE])   # for thinning
rm(cnt_dc, cyc_counts, so_dc); invisible(gc())

cyc_groups <- list(
  "DC (source)"   = \(d) d$fine %in% DC_FINE,
  "cDC"           = \(d) d$fine == "cDC",
  "pDC"           = \(d) d$fine == "pDC",
  "DC (expanded)" = \(d) d$fine %in% DC_FINE | (d$fine %in% PROLIF_LABELS & d$dc_marker),
  "Macrophage"    = \(d) d$pop %in% "Macrophage",
  "Monocyte"      = \(d) d$pop %in% "Monocyte",
  "T cell"        = \(d) d$pop %in% "T cell",
  "NK"            = \(d) d$pop %in% "NK",
  "B cell"        = \(d) d$pop %in% "B cell",
  "Other immune"  = \(d) d$lineage == "Immune" & !d$fine %in% DC_FINE,
```

```
  "Epithelial"    = \(d) d$lineage == "Epithelial",
  "Endothelial"   = \(d) d$lineage == "Endothelial",
  "Mesenchymal"   = \(d) d$lineage == "Mesenchymal")

cyc_rates <- imap_dfr(cyc_groups, \(f, g) cell_cyc[f(cell_cyc), ] %>%
                        group_by(donor) %>%
                        summarise(n = n(), k = sum(cycling), k1 = sum(n_cyc >= 1), median_umi = stats::median(umi),
.groups = "drop") %>%
                        mutate(population = g)) %>%
  inner_join(donors, by = "donor") %>%
  mutate(pct = 100 * k / n, logit = stats::qlogis((k + 0.5) / (n + 1)),
         ci_lo = 100 * map2_dbl(k, n, \(a, b) stats::binom.test(a, b)$conf.int[1]),
         ci_hi = 100 * map2_dbl(k, n, \(a, b) stats::binom.test(a, b)$conf.int[2]))


# Cells added by the expanded definition (candidate DCs inside proliferating labels;
# marker co-detection may also be transcript spillover)
dc_rescue <- cell_cyc %>% filter(fine %in% PROLIF_LABELS, dc_marker) %>%
  group_by(donor, role, fine) %>% summarise(n_added = n(), n_added_cycling = sum(cycling), .groups = "drop")
dc_marker_qc <- cell_cyc %>% filter(fine %in% DC_FINE | (fine %in% PROLIF_LABELS & dc_marker)) %>%
  group_by(role, fine) %>%
  summarise(n = n(), pct_dc_marker = 100 * mean(dc_marker), pct_cycling = 100 * mean(cycling),
            median_umi = stats::median(umi), .groups = "drop")
```

10C. DONOR MODELS: DC CYCLING, SPECIFICITY, ROBUSTNESS

```
cyc_fit <- function(d) {                                  # d: donor rows with logit, PMA_weeks, role
  d <- filter(d, role %in% c(ref_role, inj_role), n >= DC_MIN_N)
  if (sum(d$role == inj_role) < 3 || sum(d$role == ref_role) < 6) return(tibble())
  s  <- summary(stats::lm(logit ~ PMA_weeks + I(role == inj_role), data = d))
  co <- s$coefficients[3, ]; tc <- stats::qt(0.975, s$df[2])
  tibble(n_ref = sum(d$role == ref_role), n_inj = sum(d$role == inj_role),
         beta = co[["Estimate"]], lo = co[["Estimate"]] - tc * co[["Std. Error"]],
         hi = co[["Estimate"]] + tc * co[["Std. Error"]], p = co[["Pr(>|t|)"]],
         OR = exp(beta), OR_lo = exp(lo), OR_hi = exp(hi))
}

cyc_models <- cyc_rates %>% group_by(population) %>% group_modify(\(x, ...) cyc_fit(x)) %>% ungroup() %>%
  mutate(FDR = stats::p.adjust(p, "BH"),
         population = factor(population, levels = names(cyc_groups))) %>% arrange(population)

# Difference-in-differences: DC minus each comparator, per donor
did_models <- map_dfr(setdiff(names(cyc_groups), c("DC (source)", "cDC", "pDC", "DC (expanded)")), \(g) {
  w <- cyc_rates %>% filter(population %in% c("DC (source)", g)) %>%
    select(donor, population, logit, n, PMA_weeks, role) %>%
    pivot_wider(names_from = population, values_from = c(logit, n)) %>%
    filter(!is.na(.data[[paste0("logit_", g)]]), !is.na(`logit_DC (source)`))
  w %>% transmute(donor, PMA_weeks, role, logit = `logit_DC (source)` - .data[[paste0("logit_", g)]],
                  n = pmin(`n_DC (source)`, .data[[paste0("n_", g)]])) %>%
    cyc_fit() %>% mutate(comparison = paste("DC minus", g), .before = 1)
}) %>% mutate(FDR = stats::p.adjust(p, "BH"))

# Leave-one-donor-out (source and expanded definitions)
loo_dc <- map_dfr(c("DC (source)", "DC (expanded)"), \(g) {
  d <- filter(cyc_rates, population == g, role %in% c(ref_role, inj_role), n >= DC_MIN_N)
  map_dfr(d$donor, \(i) cyc_fit(filter(d, donor != i)) %>% mutate(dropped = i, population = g))
})

# Equal-depth rarefaction: each DC (source definition, UMI >= target) is reduced to
# exactly `target` molecules drawn without replacement from its full-panel pool;
# drawing the five cycle genes sequentially from the hypergeometric is exact.
# Cells below the target are excluded (retention reported per donor).
rarefy_cycle <- function(cyc, totals, depth) {
  n <- length(totals); remaining_total <- as.integer(totals)
  remaining_draw <- rep(as.integer(depth), n); detected <- integer(n)
  for (j in seq_len(nrow(cyc))) {
    good <- as.integer(cyc[j, ]); bad <- pmax(remaining_total - good, 0L)
    hit  <- stats::rhyper(n, good, bad, remaining_draw)
    detected <- detected + as.integer(hit > 0)
```

```
    remaining_total <- remaining_total - good; remaining_draw <- remaining_draw - hit
  }
  detected >= CYCLE_MIN
}
dc_cells <- filter(cell_cyc, fine %in% DC_FINE)
set.seed(42)
thin_runs <- map_dfr(THIN_UMI, \(target) {
  keep <- dc_cells$umi >= target
  sub  <- dc_cells[keep, ]; X <- dc_cyc_counts[, keep, drop = FALSE]
  retained <- dc_cells %>% group_by(donor) %>% summarise(retained = mean(umi >= target), .groups = "drop")
  fits <- map_dfr(seq_len(N_THIN), \(r) {
    sub %>% mutate(cyc = rarefy_cycle(X, umi, target)) %>%
      group_by(donor) %>% summarise(n = n(), k = sum(cyc), .groups = "drop") %>%
      inner_join(donors, by = "donor") %>%
      mutate(logit = stats::qlogis((k + 0.5) / (n + 1))) %>%
      cyc_fit() %>% mutate(rep = r)
  })
  tibble(target = target, runs = list(fits), retained = list(retained))
})
thin_summary <- thin_runs %>% mutate(s = map(runs, \(x) tibble(
  beta_median = stats::median(x$beta), beta_lo = stats::quantile(x$beta, 0.025),
  beta_hi = stats::quantile(x$beta, 0.975), frac_negative = mean(x$beta < 0),
  frac_p05 = mean(x$p < 0.05)))) %>% select(target, s) %>% unnest(s)
thin_retained <- thin_runs %>% select(target, retained) %>% unnest(retained) %>% left_join(donors, by = "donor")

cat("\n=== TABLE 10.1  Cycling fraction per donor: dendritic cells (source definition) ===\n")
print(cyc_rates %>% filter(population == "DC (source)") %>%
        select(donor, role, GA_weeks, PNA_weeks, PMA_weeks, n, k, k1, pct, ci_lo, ci_hi, median_umi) %>%
        mutate(across(c(pct, ci_lo, ci_hi), \(v) round(v, 2))) %>% arrange(role, PMA_weeks), n = Inf)
cat("\n=== TABLE 10.2  Cycling ~ PMA + injury by population (logit; specificity) ===\n")
print(cyc_models %>% mutate(across(c(beta, lo, hi, OR, OR_lo, OR_hi), \(v) round(v, 3)),
                            across(c(p, FDR), \(v) signif(v, 3))), n = Inf, width = Inf)
cat("\n=== TABLE 10.3  Difference-in-differences: DC minus comparator ===\n")
print(did_models %>% mutate(across(c(beta, lo, hi, OR), \(v) round(v, 3)), across(c(p, FDR), \(v) signif(v, 3))), n =
Inf)
cat("\n=== TABLE 10.4  Leave-one-donor-out and equal-depth thinning ===\n")
print(loo_dc %>% group_by(population) %>%
        summarise(n_refits = n(), frac_negative = mean(beta < 0), min_beta = min(beta), max_beta = max(beta),
                  n_p05 = sum(p < 0.05), .groups = "drop"))
print(thin_summary %>% mutate(across(where(is.double), \(v) round(v, 3))))
```

10D. LungMAP: PRE-SPECIFIED DC CYCLING TEST

```
# Outcome: fraction of DC nuclei (cDC + pDC) with >= 2 of the five cycle genes,
# per donor (>= LM_DC_MIN_N nuclei). Model: logit ~ BPD + PMA; one-sided
# (BPD lower), alpha 0.05; run once and reported regardless of outcome.
# Descriptive: stage estimates; other populations for context.

lm_cells_cc <- lm_raw$cells %>%
  mutate(n_cyc = rowSums(across(any_of(cyc_g), \(v) v > 0)), cycling = n_cyc >= CYCLE_MIN)
lm_cyc_groups <- list("DC (cDC + pDC)" = c("cDC", "pDC"), "cDC" = "cDC", "pDC" = "pDC",
                      "Alveolar macrophage" = "AM", "Interstitial macrophage" = "IM", "Monocyte" = "MON",
                      "T cell" = "TCell", "NK" = "NK", "B cell" = "BCell")
lm_cyc_rates <- imap_dfr(lm_cyc_groups, \(labs_, g) lm_cells_cc %>% filter(label %in% labs_) %>%
                           group_by(donor) %>% summarise(n = n(), k = sum(cycling), .groups = "drop") %>%
mutate(population = g)) %>%
  inner_join(lm_donors, by = "donor") %>%
  mutate(pct = 100 * k / n, logit = stats::qlogis((k + 0.5) / (n + 1)),
         ci_lo = 100 * map2_dbl(k, n, \(a, b) stats::binom.test(a, b)$conf.int[1]),
         ci_hi = 100 * map2_dbl(k, n, \(a, b) stats::binom.test(a, b)$conf.int[2]))

lm_cyc_fit <- function(d) {
  d <- filter(d, n >= LM_DC_MIN_N)
  if (sum(d$injury == "Injured") < 3 || sum(d$injury == "Reference") < 3) return(tibble())
  s <- summary(stats::lm(logit ~ injury + PMA_weeks, data = d)); co <- s$coefficients["injuryInjured", ]
  tc <- stats::qt(0.975, s$df[2])
  tibble(n_ctrl = sum(d$injury == "Reference"), n_bpd = sum(d$injury == "Injured"),
         beta = co[["Estimate"]], lo = co[["Estimate"]] - tc * co[["Std. Error"]],
         hi = co[["Estimate"]] + tc * co[["Std. Error"]], p_two_sided = co[["Pr(>|t|)"]],
         p_one_sided_lower = stats::pt(co[["t value"]], s$df[2]), OR = exp(beta))
```

```
}
lm_cyc_models <- lm_cyc_rates %>% group_by(population) %>% group_modify(\(x, ...) lm_cyc_fit(x)) %>% ungroup() %>%
  mutate(population = factor(population, levels = names(lm_cyc_groups))) %>% arrange(population)
lm_cyc_stage <- {
  d <- filter(lm_cyc_rates, population == "DC (cDC + pDC)", n >= LM_DC_MIN_N)
  if (nrow(d) >= 8) {
    f <- stats::lm(logit ~ stage + PMA_weeks, data = mutate(d, stage = factor(stage, levels = lm_stage_levels)))
    ci <- stats::confint(f)
    tibble(term = names(stats::coef(f)), estimate = stats::coef(f), lo = ci[, 1], hi = ci[, 2]) %>%
      filter(grepl("^stage", term))
  } else tibble()
}

cat("\n=== TABLE 10.5  LungMAP cycling by population (BPD vs control, PMA-adjusted) ===\n")
cat("PRIMARY (pre-specified): DC (cDC + pDC), one-sided P for lower cycling in BPD\n")
print(lm_cyc_models %>% mutate(across(c(beta, lo, hi, OR), \(v) round(v, 3)),
                               across(starts_with("p_"), \(v) signif(v, 3))), n = Inf, width = Inf)
print(lm_cyc_stage %>% mutate(across(where(is.double), \(v) round(v, 3))))
```

10I.1. GENES AVAILABLE PER PROGRAM

```
so_st <- GSE297945_final_complete_cniches_tniches_allsamples_2025_05_26
panel_ok <- intersect(rownames(so_st), filter(gene_lineage, immune_ratio >= 1)$gene)
xen_sets <- lapply(STATE_SETS, intersect, panel_ok)
lm_genes <- rownames(lm_raw$pb)
lm_sets  <- lapply(STATE_SETS, intersect, lm_genes)

state_genes_tbl <- imap_dfr(STATE_SETS, \(g, p) tibble(
  programme = p, n_defined = length(g),
  xenium_genes = paste(xen_sets[[p]], collapse = ", "), n_xenium = length(xen_sets[[p]]),
  n_lungmap = length(lm_sets[[p]]),
  xenium_scored = length(xen_sets[[p]]) >= MIN_GENES_STATE,
  lungmap_scored = length(lm_sets[[p]]) >= MIN_GENES_STATE))
cat("\n=== TABLE D1  Programme genes available (Xenium: panel and immune-specific) ===\n")
print(state_genes_tbl, n = Inf, width = Inf)
xen_sets <- xen_sets[lengths(xen_sets) >= MIN_GENES_STATE]
lm_sets  <- lm_sets[lengths(lm_sets) >= MIN_GENES_STATE]
```

10I.2. XENIUM DONOR PSEUDOBULKS OF DC SUBSETS

```
dc_st <- cell_cyc %>% filter(fine %in% c("cDC", "pDC"))
genes_needed <- unique(unlist(xen_sets))
cnt_all <- Seurat::GetAssayData(so_st, assay = "RNA", layer = "counts")
cnt_st  <- cnt_all[genes_needed, dc_st$cell, drop = FALSE]
# Depth excludes cell-cycle genes, so cycling cells are not penalised for their
# extra cycle-gene UMIs when compared with non-cycling cells (3B)
dc_st$umi_nc <- dc_st$umi - Matrix::colSums(cnt_all[intersect(CYCLE_GENES, rownames(cnt_all)), dc_st$cell, drop = FALSE])
rm(so_st, cnt_all); invisible(gc())

pb_state <- function(keep, group) {                    # genes x groups (counts) + total UMIs per group
  g <- factor(group[keep])
  X <- as.matrix(cnt_st[, keep, drop = FALSE] %*% Matrix::sparse.model.matrix(~ 0 + g)); colnames(X) <- levels(g)
  list(counts = X, umi = as.numeric(tapply(dc_st$umi_nc[keep], g, sum)[colnames(X)]), n =
as.integer(table(g)[colnames(X)]))
}
log2cpm_pb <- function(pb) log2(t(t(pb$counts) / pb$umi) * 1e6 + 1)

score_programmes <- function(lc, sets) {                # z across donors, mean over genes
  map_dfr(names(sets), \(p) {
    g <- intersect(sets[[p]], rownames(lc)); z <- t(scale(t(lc[g, , drop = FALSE])))
    z <- z[apply(is.finite(z), 1, all), , drop = FALSE]
    if (nrow(z) < MIN_GENES_STATE) return(tibble())
    tibble(programme = p, donor = colnames(lc), score = colMeans(z), n_genes = nrow(z))
```

```
  })
}
```

10I.3. NON-CYCLING DCs: INJURED VS REFERENCE (PMA-ADJUSTED)

```
analysis_A <- function(subset_label, keep_fine) {
  keep <- dc_st$fine %in% keep_fine & !dc_st$cycling & dc_st$role %in% c(ref_role, inj_role)
  pb <- pb_state(keep, dc_st$donor)
  ok <- colnames(pb$counts)[pb$n >= 20]
  lc <- log2cpm_pb(pb)[, ok, drop = FALSE]
  sc <- score_programmes(lc, xen_sets) %>% left_join(donors, by = "donor") %>%
    mutate(injured = role == inj_role, subset = subset_label, n_cells = pb$n[match(donor, colnames(pb$counts))])
  shifts <- sc %>% rename(Developmental_PC = score) %>% group_by(programme) %>%
    group_modify(\(x, ...) { r <- injury_shift(x); if (is.null(r)) tibble() else r }) %>% ungroup() %>%
    mutate(FDR = stats::p.adjust(p, "BH"), subset = subset_label)
  list(scores = sc, shifts = shifts)
}
resA <- list(analysis_A("cDC (non-cycling)", "cDC"), analysis_A("cDC + pDC (non-cycling)", c("cDC", "pDC")))
stateA_scores <- map_dfr(resA, "scores"); stateA_shifts <- map_dfr(resA, "shifts")

cat("\n=== TABLE D2  Non-cycling DC state, injured vs reference (PMA-adjusted; primary = cDC) ===\n")
print(stateA_shifts %>% select(subset, programme, n_ref, n_inj, shift_SD, lo, hi, p, FDR, MDE80_SD) %>%
        mutate(across(c(shift_SD, lo, hi, MDE80_SD), \(v) round(v, 2)), across(c(p, FDR), \(v) signif(v, 3))), n = Inf)
```

10I.4. REFERENCE DONORS: CYCLING VS NON-CYCLING cDCs

```
keepB <- dc_st$fine == "cDC" & dc_st$role == ref_role
grpB  <- paste(dc_st$donor, ifelse(dc_st$cycling, "cyc", "non"), sep = "___")
pbB   <- pb_state(keepB, grpB)
cpmB  <- t(t(pbB$counts) / pbB$umi) * 1e6
donB  <- sub("___.*", "", colnames(cpmB)); kindB <- sub(".*___", "", colnames(cpmB))
eligible <- intersect(donB[kindB == "cyc" & pbB$n >= 5], donB[kindB == "non" & pbB$n >= 20])

stateB <- map_dfr(eligible, \(d) map_dfr(names(xen_sets), \(p) {
  g <- xen_sets[[p]]
  tibble(donor = d, programme = p,
         diff = mean(log2((cpmB[g, paste0(d, "___cyc")] + 1) / (cpmB[g, paste0(d, "___non")] + 1))))
})) %>% left_join(donors, by = "donor")
stateB_test <- stateB %>% group_by(programme) %>%
  summarise(n_donors = n(), median_diff = stats::median(diff), n_higher_in_cycling = sum(diff > 0),
            p_wilcox = if (n() >= 5) stats::wilcox.test(diff, exact = FALSE)$p.value else NA_real_, .groups = "drop") %>%
  mutate(FDR = stats::p.adjust(p_wilcox, "BH"))

cat("\n=== TABLE D3  Reference donors: cycling minus non-cycling cDCs (log2 ratio, within donor) ===\n")
print(stateB_test %>% mutate(median_diff = round(median_diff, 3), across(c(p_wilcox, FDR), \(v) signif(v, 3))), n = Inf)
```

10I.5. LungMAP: cDC STATE, BPD VS CONTROL

```
lm_cdc_n <- lm_raw$n_labelled %>% filter(label == "cDC") %>% select(donor, n_cdc = n)
cols_c   <- grep("___cDC$", colnames(lm_raw$pb), value = TRUE)
pb_lm_c  <- lm_raw$pb[, cols_c, drop = FALSE]; colnames(pb_lm_c) <- sub("___cDC$", "", cols_c)
pb_lm_c  <- pb_lm_c[, intersect(colnames(pb_lm_c), filter(lm_cdc_n, n_cdc >= 10)$donor), drop = FALSE]
lc_lm_c  <- logcpm(pb_lm_c)
stateC_scores <- score_programmes(lc_lm_c, lm_sets) %>% left_join(lm_donors, by = "donor") %>%
  mutate(injured = injury == "Injured")
stateC_shifts <- stateC_scores %>% rename(Developmental_PC = score) %>% group_by(programme) %>%
  group_modify(\(x, ...) { r <- injury_shift(x); if (is.null(r)) tibble() else r }) %>% ungroup() %>%
  mutate(FDR = stats::p.adjust(p, "BH"))
```

```
cat("\n=== TABLE D4  LungMAP cDC state, BPD vs control (PMA-adjusted; two-sided) ===\n")
print(stateC_shifts %>% select(programme, n_ref, n_inj, shift_SD, lo, hi, p, FDR, MDE80_SD) %>%
        mutate(across(c(shift_SD, lo, hi, MDE80_SD), \(v) round(v, 2)), across(c(p, FDR), \(v) signif(v, 3))), n = Inf)
```

10I.7. PER-NUCLEUS DATA FOR LungMAP cDCs (one extra read, cached)

```
WATCH2 <- unique(c(unlist(FOLLOW_GENES), CDC1_MK, CDC2_MK, CYCLE_GENES))
LM_CDC_CACHE <- file.path(LUNGMAP_DIR, "lungmap_cdc_nuclei.rds")
cdc_rows <- lm_raw$cells %>% filter(label == "cDC") %>% select(row, barcode, donor)
cached <- if (file.exists(LM_CDC_CACHE)) readRDS(LM_CDC_CACHE) else NULL
if (!is.null(cached) && identical(cached$watch, WATCH2) && nrow(cached$cells) == nrow(cdc_rows)) {
  nuc <- cached$cells; message("LungMAP cDC nuclei: using cache ", LM_CDC_CACHE)
} else {
  message("LungMAP: reading ", nrow(cdc_rows), " cDC nuclei for per-nucleus genes (one time, ~10 min)")
  rd  <- read_h5ad_groups(LUNGMAP_H5AD, cdc_rows$row, cdc_rows$donor, watch = WATCH2)
  nuc <- rd$cells %>% left_join(cdc_rows, by = "row")
  if (LUNGMAP_WRITE_CACHE) saveRDS(list(watch = WATCH2, cells = nuc), LM_CDC_CACHE)
}
has_g <- \(g) intersect(g, names(nuc))
anypos <- \(df, g) if (length(g)) rowSums(as.matrix(df[g]) > 0) else rep(0, nrow(df))
core_lm <- has_g(MREG_CORE)
nuc <- nuc %>% mutate(n_core   = anypos(nuc, core_lm),
                      mreg_pos = n_core >= MREG_MIN,
                      strict   = if (all(c("LAMP3", "CCR7") %in% names(nuc))) LAMP3 > 0 & CCR7 > 0 else NA,
                      cycling  = anypos(nuc, has_g(CYCLE_GENES)) >= CYCLE_MIN,
                      c1 = anypos(nuc, has_g(CDC1_MK)) > 0, c2 = anypos(nuc, has_g(CDC2_MK)) > 0)

pb_cpm <- t(t(pb_lm_c) / colSums(pb_lm_c)) * 1e6
min_don <- ceiling(2 / 3 * ncol(pb_lm_c))
det_tbl <- tibble(gene = unique(c(unlist(FOLLOW_GENES), CDC1_MK, CDC2_MK))) %>%
  mutate(group = coalesce(map_chr(gene, \(g) { k <- names(FOLLOW_GENES)[map_lgl(FOLLOW_GENES, \(s) g %in% s)]; if
(length(k)) k[1] else NA_character_ }),
                          if_else(gene %in% CDC1_MK, "cDC1 marker", "cDC2 marker")),
         in_lungmap = gene %in% rownames(pb_lm_c),
         pct_cdc_nuclei = map_dbl(gene, \(g) if (g %in% names(nuc)) 100 * mean(nuc[[g]] > 0) else NA_real_),
         donors_detected = map_int(gene, \(g) if (g %in% rownames(pb_lm_c)) sum(pb_lm_c[g, ] > 0) else NA_integer_),
         mean_cpm = map_dbl(gene, \(g) if (g %in% rownames(pb_cpm)) mean(pb_cpm[g, ]) else NA_real_),
         tested = !is.na(donors_detected) & donors_detected >= min_don & group %in% names(FOLLOW_GENES),
         xenium_scorable = gene %in% unlist(xen_sets))
cat("\n=== TABLE F0  Gene detection in LungMAP cDCs (", ncol(pb_lm_c), " donors; tested if detected in >= ",
    min_don, ") ===\n", sep = "")
print(det_tbl %>% mutate(across(c(pct_cdc_nuclei, mean_cpm), \(v) round(v, 1))), n = Inf, width = Inf)
```

10I.8. GENE-LEVEL DECOMPOSITION

```
gene_lvl <- det_tbl %>% filter(group %in% names(FOLLOW_GENES)) %>% select(group, gene, tested) %>%
  mutate(fit = map2(gene, tested, \(g, ok) {
    if (!ok) return(tibble())
    d <- donor_meta %>% inner_join(tibble(donor = colnames(lc_lm_c), z = as.numeric(scale(lc_lm_c[g, ]))), by = "donor")
    shift_fit(d, "z") })) %>%
  unnest(fit, keep_empty = TRUE) %>% mutate(FDR = stats::p.adjust(p, "BH"))
cat("\n=== TABLE F1  Gene-level shifts, LungMAP cDCs, BPD vs control (PMA-adjusted) ===\n")
print(gene_lvl %>% select(group, gene, tested, shift_SD, lo, hi, p, FDR) %>%
        mutate(across(c(shift_SD, lo, hi), \(v) round(v, 2)), across(c(p, FDR), \(v) signif(v, 3))), n = Inf)
```

10I.9. MATCHED-GENE SCORES (LungMAP restricted to Xenium-scorable genes)

```
matched_sets <- lapply(xen_sets, intersect, rownames(lc_lm_c)); matched_sets <- matched_sets[lengths(matched_sets) >= 2]
matched_scores <- score_programmes(lc_lm_c, matched_sets) %>% left_join(donor_meta, by = "donor")
matched_tbl <- matched_scores %>% group_by(programme) %>% group_modify(\(x, ...) shift_fit(x, "score")) %>% ungroup() %>%
  mutate(version = "Xenium-scorable genes only", genes = map_chr(programme, \(p) paste(matched_sets[[p]], collapse = ",
"))) %>%
  bind_rows(stateC_shifts %>% mutate(version = "Full gene set", genes = map_chr(programme, \(p) paste(lm_sets[[p]],
collapse = ", ")))) %>%
  left_join(stateA_shifts %>% filter(subset == "cDC (non-cycling)") %>%
              select(programme, xenium_shift = shift_SD, xenium_lo = lo, xenium_hi = hi, xenium_p = p), by = "programme")
%>%
  select(programme, version, genes, n_ref, n_inj, shift_SD, lo, hi, p, MDE80_SD, starts_with("xenium")) %>%
arrange(programme, version)
cat("\n=== TABLE F2  LungMAP programme shifts: full vs Xenium-scorable genes (Xenium shift alongside) ===\n")
print(matched_tbl %>% select(-genes) %>% mutate(across(where(is.double), \(v) signif(v, 3))), n = Inf, width = Inf)
```

10I.11. ROBUSTNESS AND STAGE

```
rob_meas <- c(maturation = "Maturation / migration", mreg_sig = "mregDC signature")
loo_fu <- imap_dfr(rob_meas, \(l, y) map_dfr(lm_fu$donor, \(d) shift_fit(filter(lm_fu, donor != d), y) %>%
                                             mutate(dropped = d, measure = l)))
loo_sum <- loo_fu %>% group_by(measure) %>%
  summarise(min_shift = min(shift_SD), max_shift = max(shift_SD), max_p = max(p),
            n_p05 = sum(p < 0.05), n_refits = n(), .groups = "drop")

sens_don <- filter(lm_fu, n_cdc >= MIN_CDC_SENS)$donor
lc_sens  <- lc_lm_c[, sens_don, drop = FALSE]
sens_fu  <- donor_meta %>% filter(donor %in% sens_don) %>%
  left_join(score_programmes(lc_sens, lm_sets["Maturation / migration"]) %>% select(donor, maturation = score), by =
"donor") %>%
  left_join(score_programmes(lc_sens, list(m = intersect(MREG_CORE, rownames(lc_sens)))) %>% select(donor, mreg_sig =
score), by = "donor")
sens_tbl <- imap_dfr(rob_meas, \(l, y) shift_fit(sens_fu, y) %>% mutate(measure = l))

stage_fit <- function(d, y) {
  d <- d[is.finite(d[[y]]), ]; d$stage <- droplevels(d$stage)
  s <- summary(stats::lm(stats::reformulate(c("stage", "PMA_weeks"), y), data = d))
  co <- s$coefficients[grep("^stage", rownames(s$coefficients)), , drop = FALSE]; sig <- s$sigma; tc <- stats::qt(0.975,
s$df[2])
  st <- sub("^stage", "", rownames(co))
  tibble(stage = st, n = as.integer(table(d$stage)[st]), shift_SD = co[, 1] / sig,
         lo = (co[, 1] - tc * co[, 2]) / sig, hi = (co[, 1] + tc * co[, 2]) / sig, p = co[, 4])
}
stage_tbl <- imap_dfr(rob_meas, \(l, y) stage_fit(lm_fu, y) %>% mutate(measure = l))

cat("\n=== TABLE F4  Robustness: leave-one-donor-out, >= ", MIN_CDC_SENS, " cDC nuclei, BPD stage (descriptive) ===\n",
sep = "")
print(loo_sum %>% mutate(across(where(is.double), \(v) signif(v, 3))), width = Inf)
print(sens_tbl %>% select(measure, n_ref, n_inj, shift_SD, lo, hi, p) %>% mutate(across(where(is.double), \(v) signif(v,
3))))
print(stage_tbl %>% mutate(across(where(is.double), \(v) signif(v, 3))), n = Inf)
```

10I.14. SPECIFICITY ACROSS IMMUNE POPULATIONS

```
pops <- intersect(names(lm_label_pop), unique(sub(".*___", "", colnames(lm_raw$pb))))
pop_tests <- map_dfr(pops, \(lb) {
  don_ok <- lm_raw$n_labelled %>% filter(label == lb, n >= MIN_NUC_POP) %>% pull(donor)
  cols   <- paste0(don_ok, "___", lb); cols <- intersect(cols, colnames(lm_raw$pb))
  if (length(cols) < 9) return(tibble())
  pb <- lm_raw$pb[, cols, drop = FALSE]; colnames(pb) <- sub("___.*", "", cols)
  lc <- logcpm(pb)
  genes <- intersect(ACT_GENES, rownames(lc)); genes <- genes[rowSums(pb[genes, , drop = FALSE] > 0) >= ceiling(2 / 3 *
ncol(pb))]
  g_res <- map_dfr(genes, \(g) {
```

```
    d <- donor_meta %>% inner_join(tibble(donor = colnames(lc), y = as.numeric(scale(lc[g, ]))), by = "donor")
    shift_fit(d, "y") %>% mutate(feature = g)
  })
  st <- intersect(STRESS_GENES, rownames(lc)); st <- st[rowSums(pb[st, , drop = FALSE] > 0) >= ceiling(2 / 3 * ncol(pb))]
  s_res <- if (length(st) >= 2) {
    d <- donor_meta %>% inner_join(score_programmes(lc, list(Stress = st)) %>% select(donor, y = score), by = "donor")
    shift_fit(d, "y") %>% mutate(feature = "Stress / immediate-early score")
  } else tibble()
  bind_rows(g_res, s_res) %>% mutate(population = lb, n_donors = ncol(pb))
}) %>% mutate(FDR = stats::p.adjust(p, "BH"))
cat("\n=== TABLE G1  Activation genes and stress score, BPD vs control, each LungMAP immune population (BH across all)
===\n")
print(pop_tests %>% select(population, feature, n_ref, n_inj, shift_SD, lo, hi, p, FDR) %>%
        mutate(across(where(is.double), \(v) signif(v, 3))), n = Inf)
```

10I.15. cDC SHIFT ADJUSTED FOR STRESS, CYCLING AND NUCLEI NUMBER

```
st_c <- intersect(STRESS_GENES, rownames(lc_lm_c))
cdc_adj <- lm_fu %>%
  left_join(tibble(donor = colnames(lc_lm_c),
                   CD40 = if ("CD40" %in% rownames(lc_lm_c)) as.numeric(scale(lc_lm_c["CD40", ])) else NA_real_,
                   CD83 = if ("CD83" %in% rownames(lc_lm_c)) as.numeric(scale(lc_lm_c["CD83", ])) else NA_real_), by =
"donor") %>%
  left_join(if (length(st_c) >= 2) score_programmes(lc_lm_c, list(S = st_c)) %>% select(donor, stress = score)
            else tibble(donor = character(), stress = numeric()), by = "donor") %>%
  mutate(cyc_logit = lgt(k_cyc, n_nuc), log_nuclei = log(n_cdc))
adj_list <- list("None" = NULL, "Stress score" = "stress", "Cycling fraction" = "cyc_logit", "Number of cDC nuclei" =
"log_nuclei")
adj_tbl <- map_dfr(c("CD40", "CD83", "maturation"), \(y) imap_dfr(adj_list, \(cv, nm) {
  if (!is.null(cv) && all(is.na(cdc_adj[[cv]]))) return(tibble())
  shift_fit(cdc_adj, y, covar = cv) %>% mutate(outcome = y, adjusted_for = nm)
})) %>% mutate(outcome = recode(outcome, maturation = "Maturation / migration score"))
cat("\n=== TABLE G2  LungMAP cDC shift, BPD vs control, with one added covariate (PMA always included) ===\n")
print(adj_tbl %>% select(outcome, adjusted_for, shift_SD, lo, hi, p) %>% mutate(across(where(is.double), \(v) signif(v,
3))), n = Inf)
```